BÁO CÁO CHI TIẾT DANH SÁCH LỖI (BUG REPORT)

TÍNH NĂNG FR-08: THANH TOÁN (CHECKOUT) (POOL B)

- **Mã sinh viên:** 23120127
- **Họ và tên:** Dương Trọng Hòa
- **Hệ thống được kiểm thử (SUT):** EShop Platform (`http://localhost:5173` / Backend API `http://localhost:3000`)
- **Tổng số lỗi phát hiện:** 7 lỗi (1 Critical, 4 High, 2 Medium)
- **Ngày lập báo cáo:** 2026-10-18

DANH SÁCH CHI TIẾT CÁC LỖI ĐÃ PHÁT HIỆN

🚨 BUG #1: BUG-FR08-01 (CRITICAL - SECURITY / PRICE TAMPERING)

- **Tiêu đề Issue:** `[FR-08][SEC-02][CRITICAL] Lỗ hổng Gian lận giá (Price Tampering) cho phép sửa tổng tiền thanh toán qua UI và API`
- **Nhãn (Labels):** `bug`, `critical`, `security`, `FR-08`, `business-logic`
- **Mức độ nghiêm trọng (Severity):** Critical (Nghiêm trọng nhất - Lỗ hổng tài chính)
- **Mức độ ưu tiên (Priority):** Urgent / Immediate
- **Môi trường:** Chrome / Windows / EShop Local (Frontend: React, Backend: Node.js, CSDL: SQLite)

1. Mô tả lỗi

Đặc tả FR-08 quy định rõ ràng:

> *"Tổng tiền thanh toán được tính tự động từ giỏ hàng và không cho phép người dùng chỉnh sửa trực tiếp." "Backend phải tự tính lại tổng tiền; không chấp nhận giá trị total_amount do client gửi lên."*

Tuy nhiên trên thực tế:

1. **Tại Frontend (`frontend-web/src/pages/Checkout.jsx` dòng 93-102):** Ô Tổng tiền thanh toán là một thẻ `<input type="number">` cho phép người dùng tự do click vào chỉnh sửa số tiền tùy ý.
2. **Tại Backend (`backend/server.js` dòng 299-304):** API `POST /api/checkout` nhận trực tiếp tham số `total_amount` từ request body của client và lưu thẳng vào bảng `orders` trong CSDL SQLite mà không hề tự tính lại tổng tiền từ giỏ hàng.

Hậu quả: Kẻ gian có thể mua đơn hàng 2 sản phẩm cao cấp trị giá **36.000.000 ₫** nhưng chỉ cần thanh toán với số tiền **1.000 ₫** (hoặc 0 ₫) và hệ thống vẫn chấp nhận tạo đơn hàng thành công!

2. Các bước tái hiện (Steps to Reproduce)

1. Đăng nhập vào hệ thống với tài khoản `test@eshop.com` / `Test1234!`.
2. Thêm vào giỏ hàng: **iPhone 15 Pro Max** (30.000.000 ₫) và **Tai nghe AirPods Pro 2** (6.000.000 ₫). Tổng tiền thực tế: 36.000.000 ₫.
3. Điều hướng tới trang Checkout (`http://localhost:5173/checkout`).

4. Tại ô "Tổng tiền thanh toán (VND)", xóa số tiền 36.000.000 ₫ đi và nhập `1000`.
5. Nhấn nút **"Xác Nhận Thanh Toán"**.
6. Kiểm tra CSDL SQLite bảng `orders`.

3. Kết quả mong đợi (Expected Result)

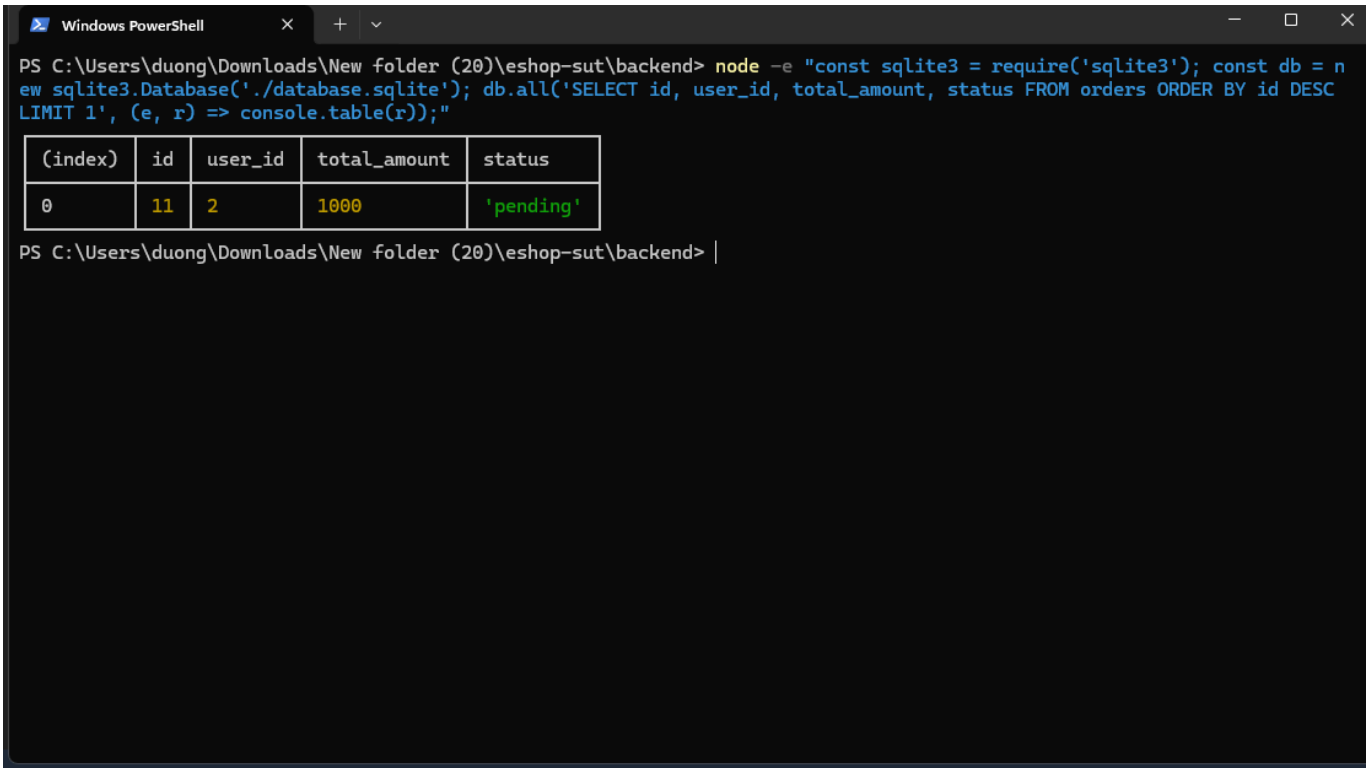
- Ô tổng tiền trên giao diện phải bị vô hiệu hóa (disabled / read-only).
- Backend bắt buộc phải tự tính lại tổng tiền từ các sản phẩm trong CSDL; nếu phát hiện giá trị gửi lên không khớp phải từ chối ngay (HTTP 400 Bad Request).

4. Kết quả thực tế (Actual Result)

Giao diện cho phép nhập số tùy ý. Backend tạo đơn hàng thành công (HTTP 200) và CSDL SQLite ghi nhận đơn hàng với `total_amount = 1000 ₫`.

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** `EXE-12_Bug_Critical_Price_Tampering.png`



🐛 BUG #2: BUG-FR08-02 (High - State Transition / Data Integrity)

- **Tiêu đề Issue:** `[FR-08][BUG] Giỏ hàng không được xóa sạch sau khi thanh toán thành công`
- **Nhãn (Labels):** `bug`, `high`, `FR-08`, `data-integrity`
- **Mức độ nghiêm trọng (Severity):** High
- **Mức độ ưu tiên (Priority):** High
- **Môi trường:** Chrome 120 / Windows 11 / EShop Local (Frontend: React, Backend: Node.js)

1. Mô tả lỗi

Đặc tả FR-08 quy định quy tắc hậu điều kiện (Post-condition):

> "*Sau thanh toán thành công, giỏ hàng được xóa.*"

Tuy nhiên, sau khi người dùng hoàn tất thanh toán và nhận thông báo thành công, giỏ hàng của người dùng vẫn giữ nguyên các sản phẩm cũ. Khi người dùng quay lại trang chủ hoặc vào trang Cart, các mặt hàng đã mua vẫn hiển thị trong giỏ, gây trải nghiệm tệ và nguy cơ người dùng đặt trùng đơn hàng.

Nguyên nhân mã nguồn:

- Trong `frontend-web/src/pages/Checkout.jsx` dòng 61, hàm `handleCheckout` chỉ gọi `setSuccess(true)` mà bỏ quên việc gọi hàm `clearCart()`.
- Trong `backend/server.js` dòng 297-309, API `POST /api/checkout` không xóa mảng `userCarts[userId]`.

2. Các bước tái hiện (Steps to Reproduce)

1. Thêm sản phẩm vào giỏ hàng và tiến hành thanh toán thành công tại `http://localhost:5173/checkout`.
2. Màn hình hiển thị "Thanh toán thành công!".
3. Nhấp vào nút "Quay lại trang chủ" hoặc truy cập đường dẫn `http://localhost:5173/cart`.
4. Quan sát số lượng và danh sách sản phẩm trong giỏ hàng.

3. Kết quả mong đợi (Expected Result)

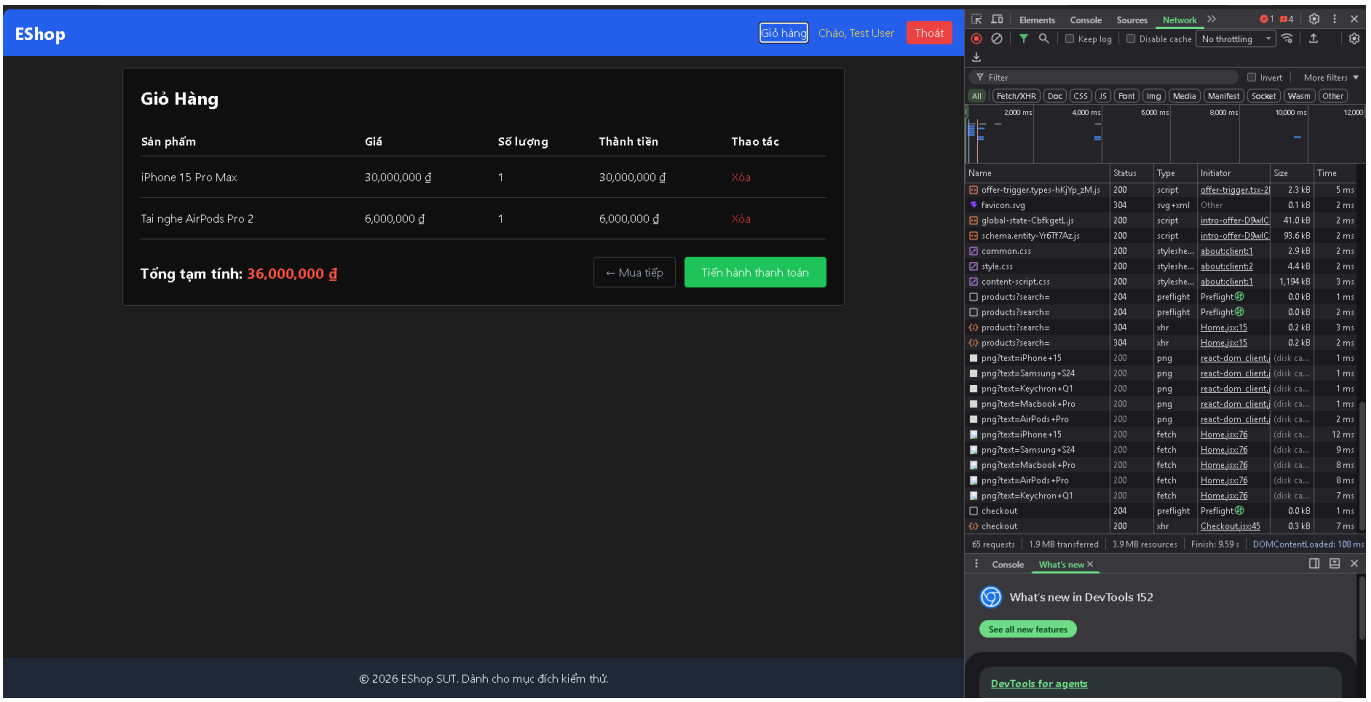
Giỏ hàng phải được làm trống (`cart.length = 0`), hiển thị thông báo "Giỏ hàng của bạn đang trống".

4. Kết quả thực tế (Actual Result)

Giỏ hàng vẫn còn nguyên các sản phẩm vừa mua và số lượng không thay đổi.

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** `EXE-05_Bug_Cart_Not_Cleared_After_Checkout.png`



🐛 BUG #3: BUG-FR08-03 (High - Missing Mandatory Field & Validation)

- **Tiêu đề Issue:** `[FR-08][FR-22][BUG] Form Checkout thiếu hoàn toàn trường Địa chỉ giao hàng và Backend chấp nhận lưu địa chỉ rỗng`
- **Nhãn (Labels):** `bug`, `high`, `FR-08`, `FR-22`, `validation`
- **Mức độ nghiêm trọng (Severity):** High
- **Mức độ ưu tiên (Priority):** High
- **Môi trường:** Chrome 120 / Windows 11 / EShop Local (Frontend: React, Backend: Node.js)

1. Mô tả lỗi

Địa chỉ giao hàng là thông tin bắt buộc thiết yếu để giao hàng. Theo FR-22: "*Tất cả các trường bắt buộc phải có dấu `*` bên cạnh nhãn*". Tuy nhiên:

1. Giao diện trang `Checkout.jsx` hoàn toàn không có trường nhập "Địa chỉ giao hàng", cũng không kế thừa và hiển thị địa chỉ mặc định từ hồ sơ người dùng (FR-04).
2. Khi gọi API `POST /api/checkout`, Backend `server.js` không kiểm tra trường `shipping_address`, cho phép client gửi chuỗi rỗng `""` hoặc bỏ trống và lưu đơn hàng không có địa chỉ giao hàng vào CSDL.

2. Các bước tái hiện (Steps to Reproduce)

1. Truy cập trang `http://localhost:5173/checkout`.
2. Quan sát toàn bộ giao diện form đặt hàng → Không thấy ô nhập Địa chỉ giao hàng.
3. Gửi request API `POST /api/checkout` với body: `{"items": [...], "total_amount": 36000000, "shipping_address": ""}`.
4. Kiểm tra CSDL SQLite bảng `orders`.

3. Kết quả mong đợi (Expected Result)

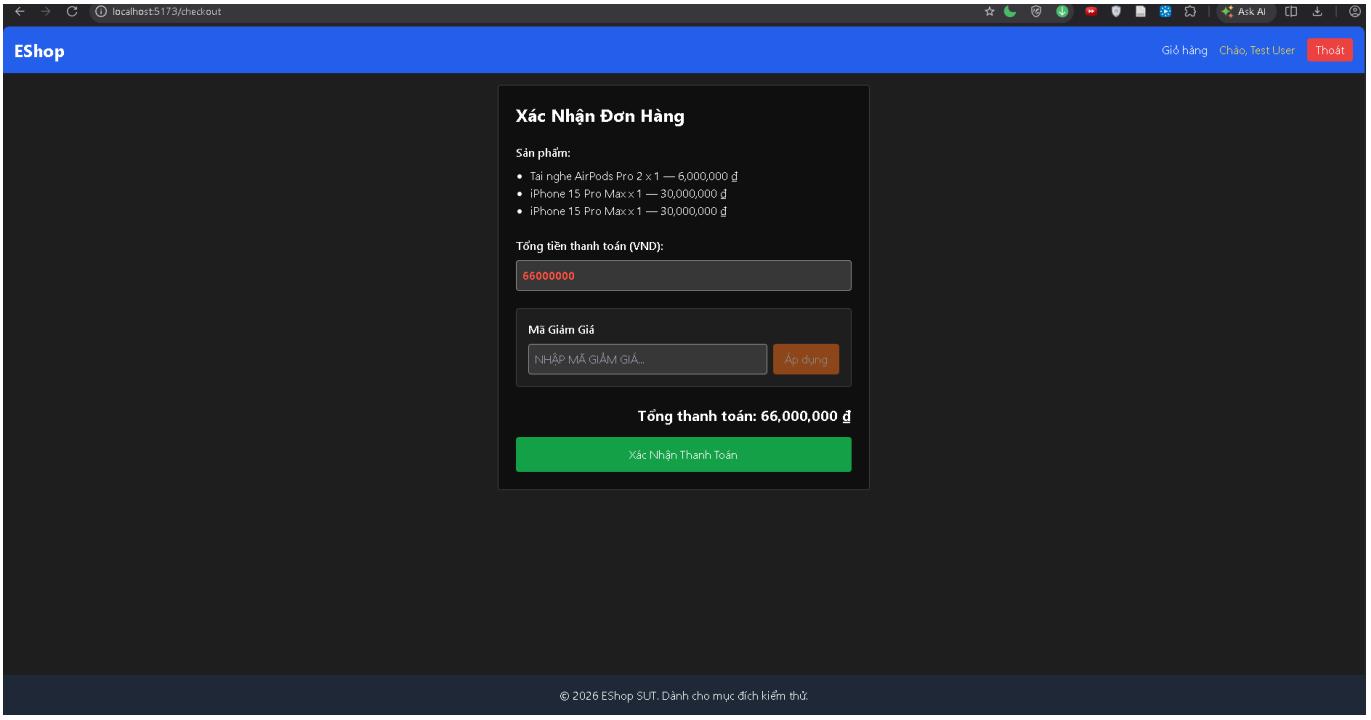
Giao diện phải có ô nhập Địa chỉ giao hàng (có dấu * đỏ). Nếu để trống, hệ thống phải chặn lại và hiển thị lỗi ở phía trên nút submit.

4. Kết quả thực tế (Actual Result)

Giao diện thiếu hoàn toàn ô nhập địa chỉ. Backend chấp nhận tạo đơn hàng thành công và lưu địa chỉ rỗng vào CSDL SQLite.

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** `EXE-03_Error_Missing_Shipping_Address_Field.png`



🐛 BUG #4: BUG-FR08-04 (Medium - GUI Standard FR-21)

- **Tiêu đề Issue:** `[FR-08][FR-21][BUG] Trang Checkout không có thẻ <h1> mô tả tiêu đề trang (đang dùng <h2>)`
- **Nhãn (Labels):** bug`, `medium`, `FR-08`, `FR-21`, `gui
- **Mức độ nghiêm trọng (Severity):** Medium
- **Mức độ ưu tiên (Priority):** Medium
- **Môi trường:** Chrome 120 / Windows 11 / EShop Local (Frontend: React, Backend: Node.js)

1. Mô tả lỗi

Quy chuẩn giao diện chung FR-21 quy định rõ ràng:

> *"Tiêu đề trang: Mỗi trang có đúng 1 thẻ `<h1>` mô tả nội dung trang."*

Tuy nhiên, tại file `frontend-web/src/pages/Checkout.jsx` dòng 80, tiêu đề "Xác Nhận Đơn Hàng" được lập trình bằng thẻ `<h2>`:

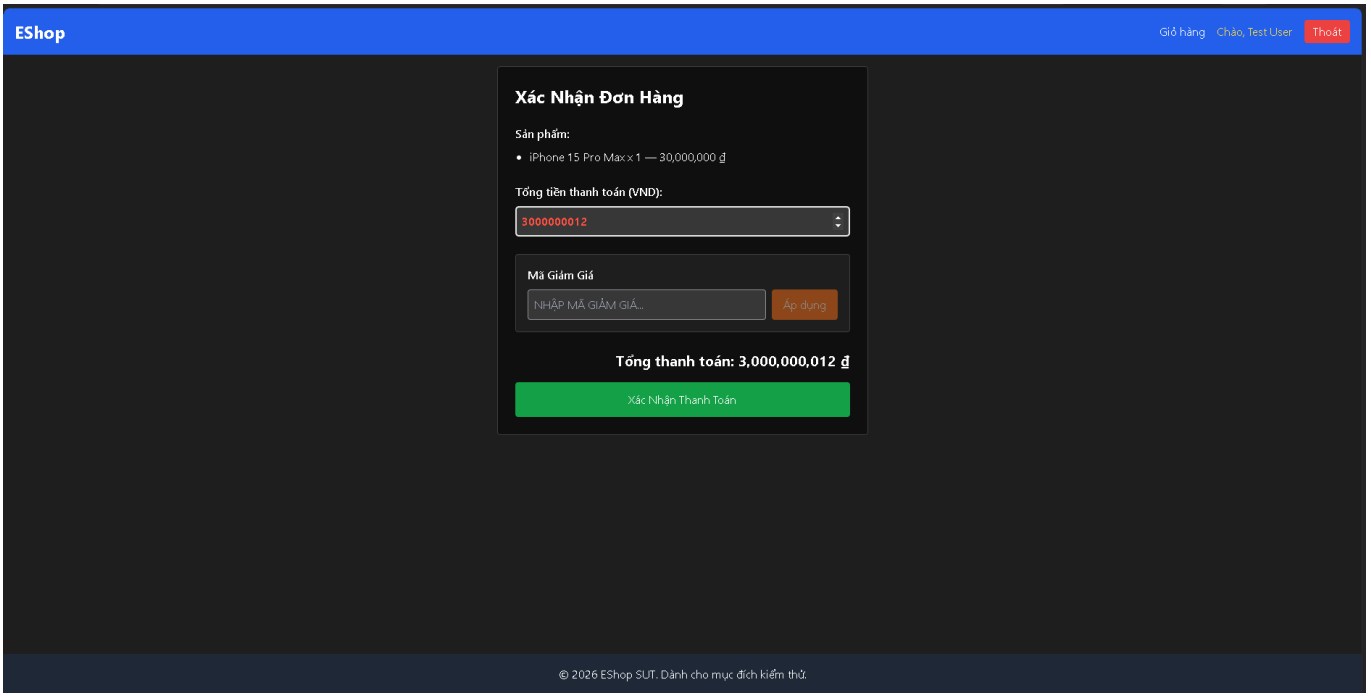
2. Các bước tái hiện (Steps to Reproduce)

3. Kết quả mong đợi (Expected Result)

4. Kết quả thực tế (Actual Result)

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** `EXE-01_GUI_TotalAmount_And_H1_Check.png`



🦎 BUG #5: BUG-FR08-05 (High - Business Logic / Empty Cart Checkout)

- **Tiêu đề Issue:** `[FR-08][BUG] Hệ thống cho phép đặt hàng khi giỏ hàng hoàn toàn trống (items.length = 0) với tổng tiền 0 ₫`
- **Nhãn (Labels):** `bug`, `high`, `FR-08`, `boundary`
- **Mức độ nghiêm trọng (Severity):** High
- **Mức độ ưu tiên (Priority):** High
- **Môi trường:** Chrome 120 / Windows 11 / EShop Local (Frontend: React, Backend: Node.js)

1. Mô tả lỗi

Tại ranh giới kiểm thử Min - 1 của giỏ hàng (`items.length = 0`), người dùng không thể thực hiện thanh toán nếu không có sản phẩm nào. Tuy nhiên, Backend tại `backend/server.js` không kiểm tra số lượng phần tử của mảng `items`, cho phép client gửi request checkout với mảng rỗng và `total_amount = 0`. Backend vẫn trả về HTTP 200 OK và tạo một đơn hàng rác 0 ₫ vào CSDL SQLite.

2. Các bước tái hiện (Steps to Reproduce)

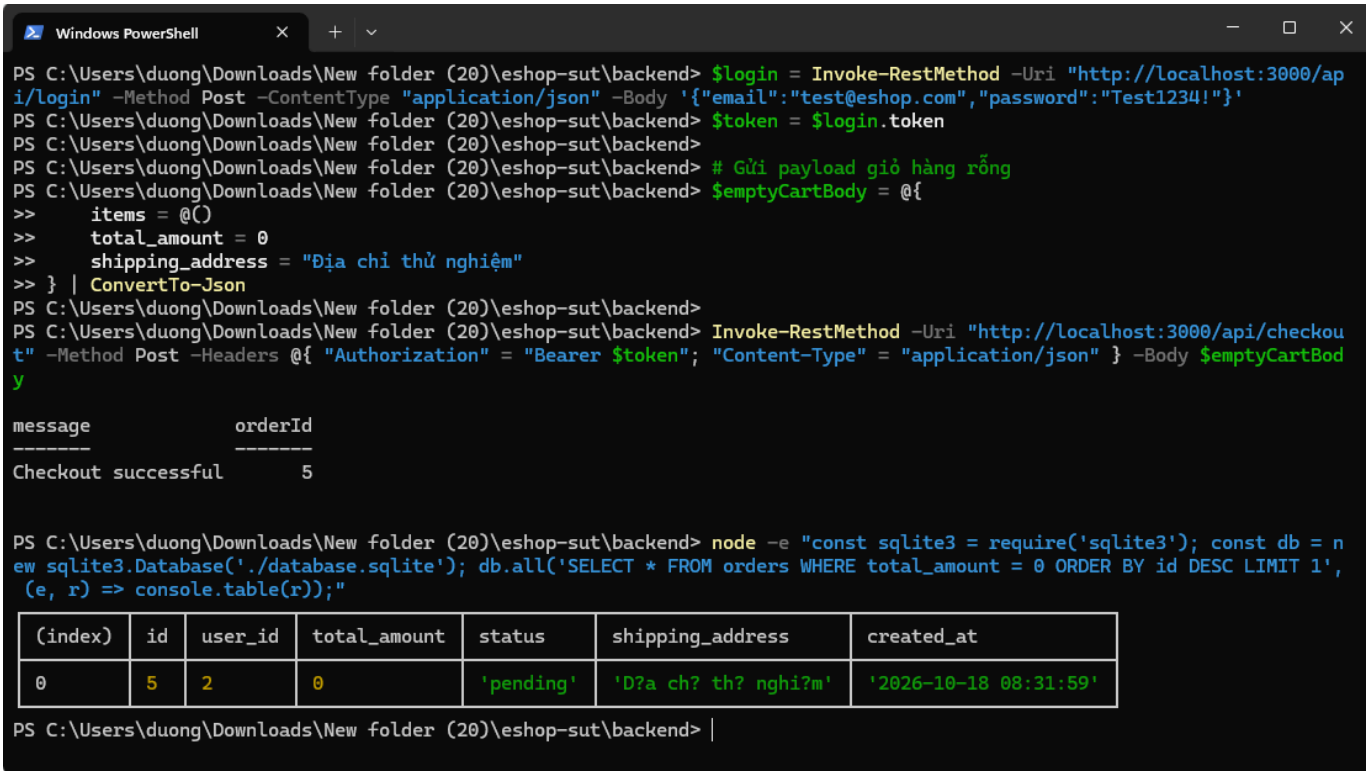
1. Gửi request `POST /api/checkout` với payload:

3. Kết quả mong đợi (Expected Result)

4. Kết quả thực tế (Actual Result)

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** `EXE-07_Bug_BVA_Empty_Cart_Accepted.png`



🐛 BUG #6: BUG-FR08-06 (High - Data Integrity / Negative Quantity Accepted)

- **Tiêu đề Issue:** `[FR-08][BUG] Backend chấp nhận tạo đơn hàng với sản phẩm có số lượng âm (quantity = -2)`
- **Nhãn (Labels):** `bug`, `high`, `FR-08`, `data-integrity`
- **Mức độ nghiêm trọng (Severity):** High
- **Mức độ ưu tiên (Priority):** High
- **Môi trường:** Chrome 120 / Windows 11 / EShop Local (Frontend: React, Backend: Node.js)

1. Mô tả lỗi

Đặc tả nghiệp vụ quy định số lượng sản phẩm đặt mua bắt buộc phải là số nguyên dương (≥ 1). Backend API `POST /api/checkout` hoàn toàn bỏ qua việc xác thực thuộc tính `quantity` của từng mặt hàng, chấp nhận đơn hàng có `quantity = -2` và trả về kết quả thành công, gây sai lệch nghiêm trọng dữ liệu tồn kho và kế toán.

2. Các bước tái hiện (Steps to Reproduce)

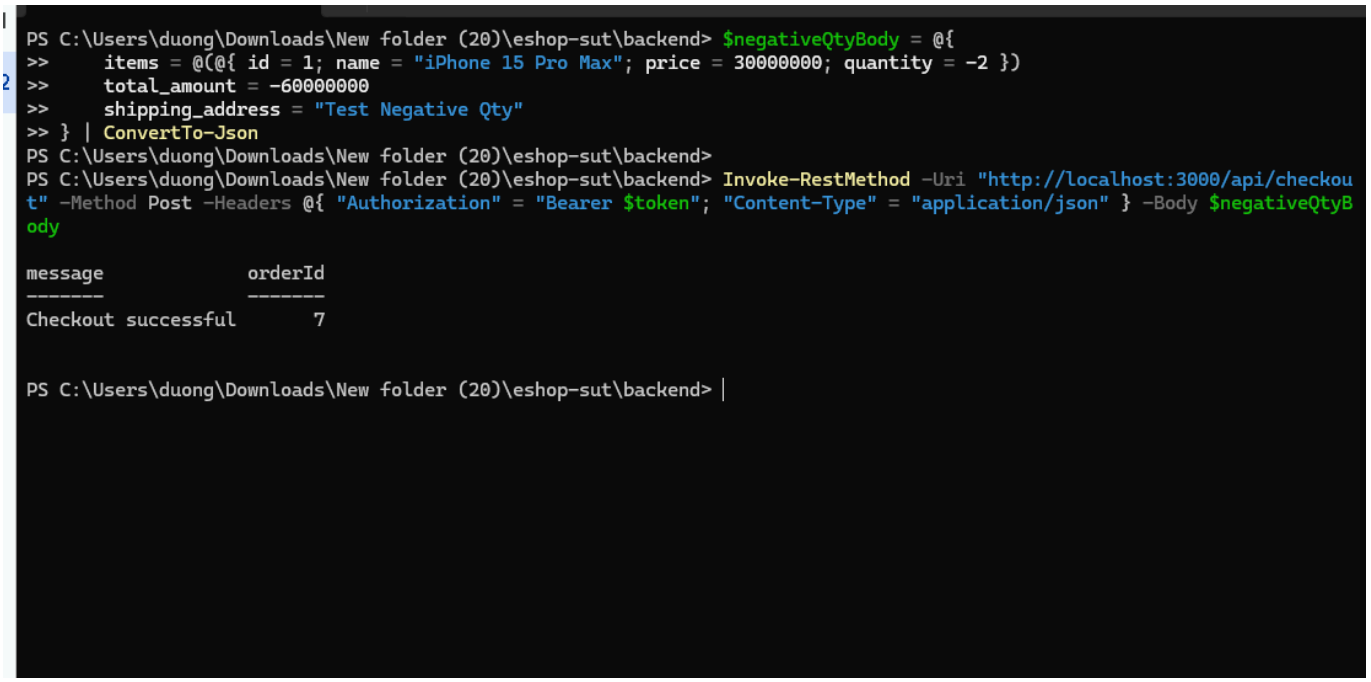
1. Gửi request `POST /api/checkout` với item có số lượng âm:

3. Kết quả mong đợi (Expected Result)

4. Kết quả thực tế (Actual Result)

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** `EXE-08_Bug_Negative_Quantity_Accepted.png`



🐛 BUG #7: BUG-FR08-07 (Medium - Boundary Validation / Address > 255 Chars)

- **Tiêu đề Issue:** `[FR-08][BUG] Không giới hạn độ dài tối đa 255 ký tự cho trường Địa chỉ giao hàng`
- **Nhãn (Labels):** `bug`, `medium`, `FR-08`, `boundary`
- **Mức độ nghiêm trọng (Severity):** Medium
- **Mức độ ưu tiên (Priority):** Medium
- **Môi trường:** Chrome 120 / Windows 11 / EShop Local (Frontend: React, Backend: Node.js)

1. Mô tả lỗi

Tại ranh giới kiểm thử Max + 1 (256 ký tự), hệ thống không có cơ chế chặn giới hạn độ dài chuỗi địa chỉ giao hàng. CSDL SQLite lưu trữ kiểu TEXT không ràng buộc và Backend không kiểm tra `shipping_address.length <= 255`, cho phép lưu trữ chuỗi vượt quá giới hạn thiết kế chuẩn (VARCHAR 255).

2. Các bước tái hiện (Steps to Reproduce)

1. Tạo chuỗi 256 ký tự: `"A" * 256`.
2. Gửi request `POST /api/checkout` với `shipping_address` là chuỗi 256 ký tự trên.
3. Quan sát kết quả trong CSDL SQLite.

3. Kết quả mong đợi (Expected Result)

Backend từ chối (HTTP 400 Bad Request) hoặc cắt bớt độ dài tối đa 255 ký tự.

4. Kết quả thực tế (Actual Result)

Backend chấp nhận và lưu toàn bộ chuỗi 256 ký tự vào CSDL với mã HTTP 200 OK.

5. Bằng chứng hình ảnh (Screenshot)

- **Ảnh minh chứng:** EXE-11_Bug_BVA_Address_256Chars_Accepted.png

